TaskFlow Backend System Documentation

1. Abstract

The TaskFlow API is a robust backend service designed for a SaaS-based Task Management and Collaboration Web Application, heavily inspired by platforms such as Trello. Built exclusively on the Node.js and Express.js runtime within the context of a MERN (MongoDB, Express, React, Node) stack, this system provides foundational RESTful endpoints required to orchestrate a Kanban-style management platform. The primary purpose of this API is to deliver scalable workflows for user authentication, stateful board manipulations, dynamic task delegation, and stringent data perseverance utilizing MongoDB.

2. Introduction

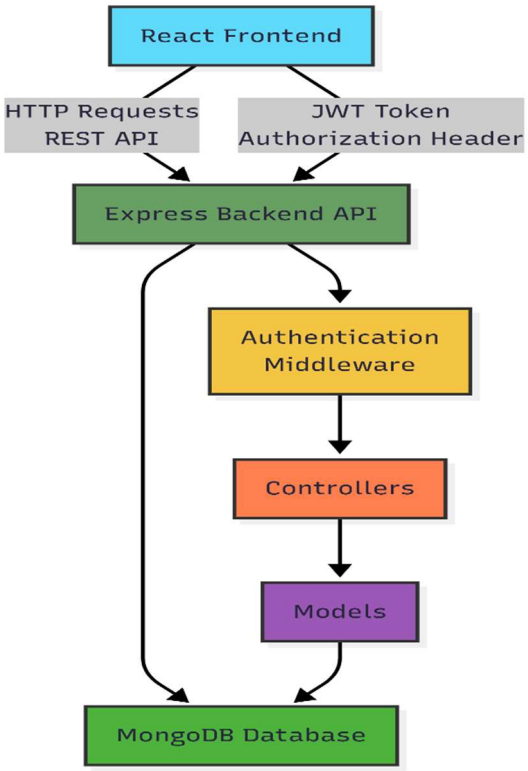
The backend serves as the centralized data processing and authentication layer for the broader TaskFlow ecosystem. Through the integration of MongoDB for persistent schema-based storage, JSON Web Tokens (JWT) for stateless identity verification, and Express.js for efficient request routing, the architecture guarantees a cohesive and responsive application framework. The overarching objective of the backend system is to ensure absolute data integrity, securely authorize user interactions with boards and tasks, and expose a scalable, logical application programming interface (API) suitable for frontend consumption.

3. System Architecture

TaskFlow leverages a client-server architecture communicating exclusively via stateless HTTP REST APIs. The step-by-step request lifecycle unfolds as follows:

1. **Request Interception:** An incoming HTTP request connects to the Express server initialized in server.js.
2. **Global Middleware Handling:** Requests are pre-processed by global middleware—cors resolves Cross-Origin constraints, express.json() parses payload bodies, and morgan logs the diagnostic output.
3. **Route Matching:** The router matches the URI to specific grouped endpoint configurations (e.g., /api/tasks).
4. **Authentication (protect):** For secured routes, the authentication middleware intercepts the execution, parses the Authorization header, mathematically verifies the JWT using a secret, and retrieves the executing User document from the database.
5. **Data Validation:** Endpoints designed for mutations implicitly funnel through Joi schemas via the validate middleware to ensure payload shape strictness.

6. **Controller Logic Execution:** The request arrives at the designated Controller function, executing dedicated business logics securely isolated from routing rules.
7. **Database Interaction:** The controller invokes Mongoose Model methods to perform Asynchronous CRUD actions mapping to MongoDB collections.
8. **Response / Exception Funneling:** Success responses culminate in outgoing JSON payloads. Conversely, synchronous validation errors or asynchronous unhandled promise rejections trigger throw new Error(), which bubbles seamlessly down to a centralized global error-handler.



4. Backend Structure

The codebase strictly adheres to a standard Separation of Concerns (SoC) methodology, systematically dividing module responsibilities into discrete directories:

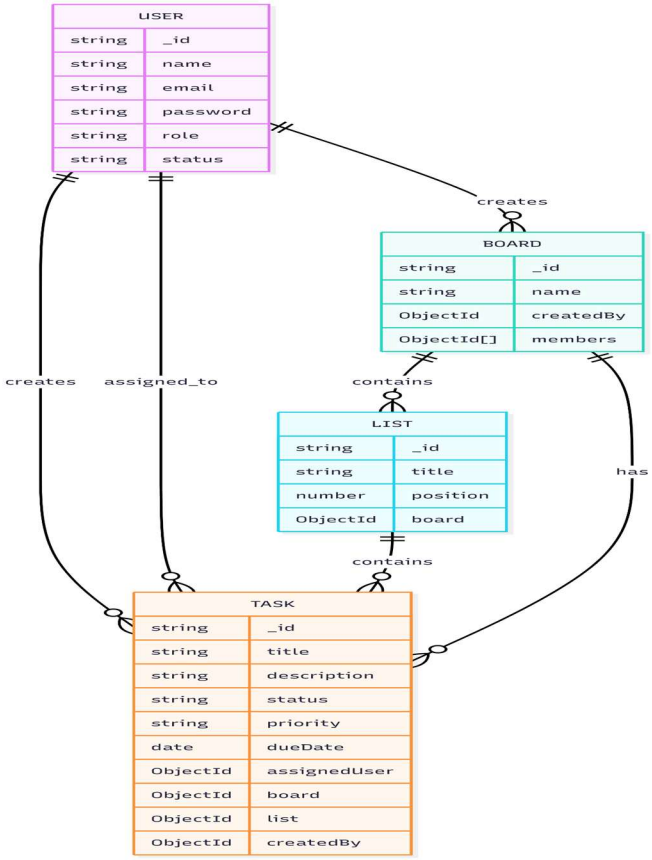
1. **server.js:** Functions as the primary entry point. Orchestrates pipeline bindings, mounts routers, connects to the database, and begins the main server listener.
2. **config/:** Houses external connections and environmental parameters. Exclusively contains db.js which manages asynchronous integrations with the MongoDB instance.
3. **controllers/:** Encapsulates core business intelligence logic. Actions are abstracted out of routing, interfacing heavily with model layers to return processed formats to the endpoints.

4. **models/:** Contains strictly typed Mongoose Database Schemas. Defines the shape, validations, enumerations, default values, and relational references defining NoSQL collections.
5. **routes/:** The network dispatch layer mapping HTTP verbs and URLs logically explicitly toward the requisite middleware and controller instances.
6. **middleware/:** Modular interceptor functions designed for reusability. Modules such as authMiddleware.js, errorMiddleware.js, and validationMiddleware.js sit here to pre-process or post-process operations consistently.
7. **utils/:** Dedicated to deterministic helper functions. Notably holds token cryptography (generateToken.js) and payload blueprints mapping constraints via Joi libraries (validationSchemas.js).

5. Database Design

The schema structures represent relational intersections typical within non-relational configurations:

1. **User (User.js):**
   - Fields: name, email (Unique, regex validated), password (Selected exclusively when needed), role (Admin/Member default restrictions), status (Active/Inactive default).
   - Relationships: Indirectly referenced heavily throughout the architecture for tracking entities creation strings.
2. **Board (Board.js):**
   - Fields: name (Max length rigidly defined limit).
   - Relationships: createdBy (References User object ID), members (An array representing many-to-many relationship mapping User Object IDs authorized to interact).
3. **List (List.js):**
   - Fields: title, position (Dictates UI rendering hierarchies intrinsically).
   - Relationships: board (Absolute reference constraining lists statically towards specific boards).
4. **Task (Task.js):**
   - Fields: title, description, status (Enum restricted: Todo, In Progress, Review, Done), priority (Enum restricted: High, Medium, Low), dueDate.
   - Relationships: Integrates vertically indexing assignedUser (User Ref), board (Board Ref), list (List Ref), and logically traces origin createdBy (User ref).

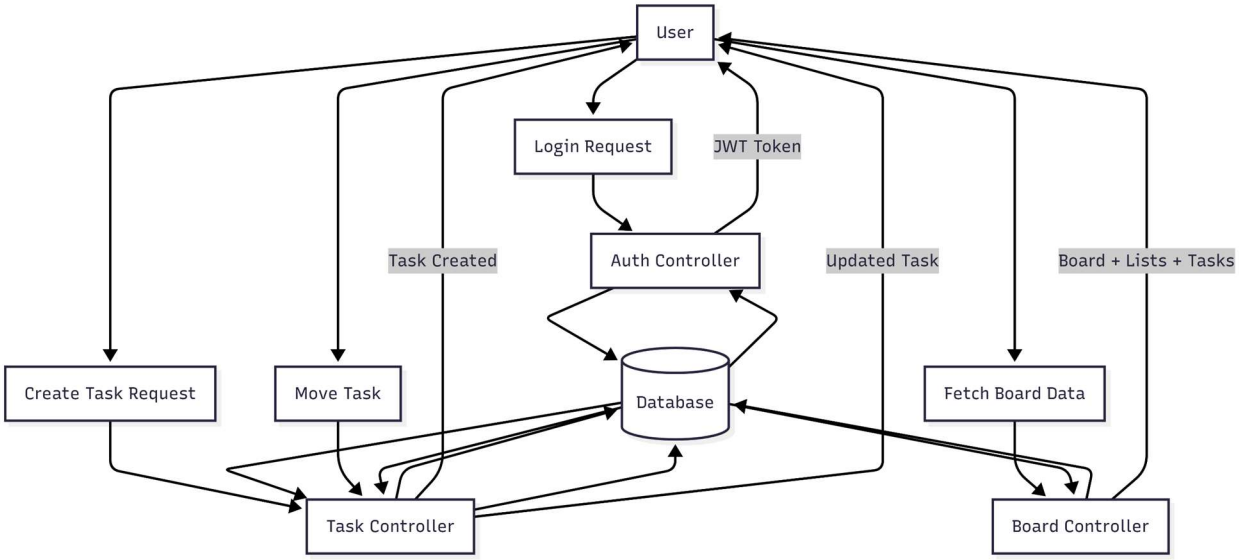


6. API Design

Below details all programmatic routes configured directly within the code logic:

1. **Auth Endpoints (/api/auth)**
   - POST /register: Registers a new system user. Validates name, email, and password via Joi. Hashes password using bcrypt prior to database insert. Returns user context and JWT.
   - POST /login: Authenticates an existing user. Pulls explicitly hidden password records and compares cryptographic hashes. Remits authentication token and profile.
2. **User Endpoints (/api/users) -** All strictly protected via JWT
   - GET /: Fetches total system users formatted in descending chronological order via creation date. Dynamically aggregates internal task assignments specific per profile.
   - GET /:id: Fetches precise user detail while computing targeted task assignment counts.
   - PUT /:id: Restricted Admin Only. Mutates system access states adjusting standard array selections via Role (admin, member) and Status (Active, Inactive).
   - DELETE /:id: Restricted Admin Only. Destroys an existing user profile explicitly preventing orphaned data by force moving user's current task array internally to the admin issuing the deletion.

3. **Board Endpoints (/api/boards)** - All strictly protected via JWT
    - GET /: Fetches boards mapping where user equals createdBy origin or explicitly resides within the internal members array.
    - GET /dashboard/overview: Performs wide aggregations producing metric calculations identifying count volumes representing tasks completed, metrics pending, and active user tallies.
    - GET /:id: Fetches deeply populated board details explicitly rendering relative nested List models correctly ordered by predefined position index mapping against active tasks representations.
    - POST /: Constructs a new Board namespace automatically binding user permissions sequentially bootstrapping out 4 explicit default Lists asynchronously ("Todo", "In Progress", "Review", "Done").
    - PATCH /:id/members/add: Interrogates existing member arrays guaranteeing zero duplication adding discrete target User ID.
    - PATCH /:id/members/remove: Mutates the membership object completely filtering specifically queried active arrays cleanly omitting deleted representations.
4. **List Endpoints (/api/lists)** - All strictly protected via JWT
    - POST /: Binds new List categorizations vertically within parsed Board reference ensuring dynamically allocated maximum length hierarchical positioning rules.
    - GET /board/:boardId: Returns list topologies bound hierarchically ascending mapped relative position arrays.
    - PUT /:id: Mutates nomenclature or indexing hierarchy coordinates within a distinct list object.
    - DELETE /:id: Deletes list target. Evaluates logic intelligently tracking task dependencies—if the target is not the primary "Todo" list, it defensively reparents active tasks back to the default "Todo" column explicitly avoiding task orphan data losses.
5. **Task Endpoints (/api/tasks)** - All strictly protected via JWT
    - POST /: Correlates valid list arrays natively binding creation values inherently normalizing matching textual rules automatically calculating strict status outputs.
    - GET /: Dynamically filters querying parameters implicitly handling queries relative to active filters (boardId, listId, status) strictly returning results chronologically.
    - PUT /:id: Generic update mapping modifying textual constraints validating specific ENUM matching states appropriately executing.
    - PATCH /:id/move: Advanced specialized logic bridging list associations mutating nested parent IDs natively coercively converting string statuses matching list target definitions.
    - DELETE /:id: Verifies precise MongoDB Object IDs explicitly executing a teardown query terminating task.



7. Controllers Logic

The core architecture applies separation between the transport logic framework and exact DB manipulations heavily within controllers:

1. **Board Controller (createBoard):** Automatically bootstraps predefined columns dynamically via Promise.all parallel array insertions drastically simplifying front-end setup mechanics natively within route resolution.
2. **List Controller (deleteList):** Acts as a safe deletion mechanism capturing lists that encapsulate active tasks. It dynamically locates a substitute "Todo" list and structurally modifies references preventing dangling reference bugs cleanly without permanent unrequested destructive wipes.
3. **Task Controller Logic (moveTask, createTask):** Relies upon a helper function normalizeStatusFromListTitle natively synchronizing specific data representations matching arbitrary UI headers logically towards fixed constraints mapping structural models implicitly preventing data de-sync.
4. **User Controller (deleteUser):** Exhibits sophisticated defensive programming principles reassigning tasks systematically specifically protecting against dangling pointer scenarios referencing administrative IDs actively performing task purges securely.

8. Authentication System

Stateless user authentication is centrally managed utilizing JSON Web Tokens:

1. Form fields are strictly validated implicitly ensuring correct shapes structurally minimizing backend injection.
2. During Registration, bcryptjs utilizes a high salt magnitude explicitly wrapping password hashing into the User payload before inserting the payload systematically into MongoDB.
3. Successful validation yields an encrypted JWT combining the _id configurations tied to environment secrets outputting 7-day expiration constraints securely.

4. Clients subsequently assert operational privileges statically passing payloads mapping exactly inside internal HTTP Authorization: Bearer <token> headers seamlessly.

9. Middleware

Modular pre-processing filters govern secure boundaries implicitly handling logic constraints explicitly:

1. **protect:** Gatekeeper evaluating JSON tokens explicitly ensuring presence strictly analyzing signature computationally binding resulting User Object implicitly to req.user.
2. **adminOnly:** Layered access control asserting execution roles matching static admin constants actively resolving permission logic strictly forbidding normal members from performing destructive endpoints.
3. **errorMiddleware (notFound, errorHandler):** Configures application catchall frameworks analyzing req.originalUrl mismatches intercepting errors globally and formatting standard messages conditionally obscuring stack traces cleanly in production environments natively.
4. **validationMiddleware (validate):** Evaluates payload bodies structurally forcing abortion sequences natively through abortEarly: true configurations preventing extensive backend computation aggressively.

10. Data Flow

Request pipelines function iteratively processing strict paths logically evaluated:

1. **Create Task Flow:**
    - Validates standard properties (title, boardId, listId, assignedUser) checking explicit structural constraints.
    - Queries exact List definition mathematically determining actual category states dynamically involving a helper mapping function (normalizeStatusFromListTitle).
    - Inserts document directly applying contextual parameters internally returning an unpopulated object dynamically.
2. **Move Task Flow:**
    - Validates Task extraction bindings resolving internal structural models implicitly referencing Target lists natively parsing references.
    - Synthesizes overwrites actively re-assigning nested list pointers calculating exact logical status mapping structural rules.
    - Persists change resolving immediately into a re-populated Task.findById() return object integrating expanded UI data directly recursively.

11. Error Handling

The backend features an explicit centralized capture mechanism utilizing internal try-catch blocks heavily applied across asynchronous layers natively. Upon catching a failure state, the

implementation maps cleanly towards next(error) callback chains routing correctly down into the server.js globally defined errorHandler. By standardizing structural variables, developers easily dispatch arbitrary errors setting res.status() explicit codes matching throw new Error(...) formats sequentially.

12. Limitations / Bugs

An analytical review surfaces the following logical limitations inherently present inside algorithmic mechanisms:

- **Authorization Bypass (Critical Scope):** The authMiddleware.js contains rigorous RBAC constraints explicit to internal structures natively correctly designed (authorizeBoardAccess, authorizeListAccess, authorizeTaskAccess). However, current endpoint registrations (boardRoutes.js, taskRoutes.js) completely omit these mounting functions sequentially injecting only protect. This effectively authorizes any general logged-in user to mutate task definitions arbitrarily regardless of their exact Board Membership assuming they structurally acquire explicit MongoDB reference identifiers explicitly routing directly securely.
- **Asynchronous List Deletion Cascades:** Resolving target deletions mapping dynamically falling backwards onto string variables (title: "Todo") natively evaluates structurally towards exact textual mapping implicitly meaning custom renamed columns internally break routing fallback algorithms implicitly structurally dropping Tasks permanently unexpectedly explicitly natively.
- **Implicit Scale/Memory Constraints:** Broad fetch mechanisms (e.g., getTasks, getUsers, getBoards) lack computational aggregation safeguards actively resolving complete collection matches completely omitting implicit .limit()/.skip() boundaries routing gigabyte scale unprocessable data towards internal V8 rendering limits systematically conceptually.
- **Unwrapped Database Migrations**: Complex Board allocations inherently dynamically mapping 4 recursive lists operate entirely independently of atomic multi-document internal withTransaction() sessions. Should step 3 fail actively, standard code pipelines systematically introduce incomplete orphan boards natively cleanly conceptually mapped structurally inside collections accurately globally natively inherently structurally.

13. Conclusion

The TaskFlow Backend explicitly demonstrates scalable, precisely constructed MVC framework integrations matching highly resilient structural programmatic practices effectively mapping modern Mongoose relational mechanisms appropriately. By systematically separating transport parameters mathematically from logic structures properly handling implicit object aggregations,

the codebase efficiently satisfies strict functional constraints while retaining scalable organizational patterns.

14. Future Scope

- Secure Authorization Implementation: Enforcing strict target middlewares actively inserting missing bindings matching Board Membership properly.
- Database Transactability: Synchronizing core object creations internally through transactional bindings fundamentally shielding internal structural consistency properly.
- Performance Enhancements: Implementing extensive computational pagination controls properly filtering and sorting data computationally.